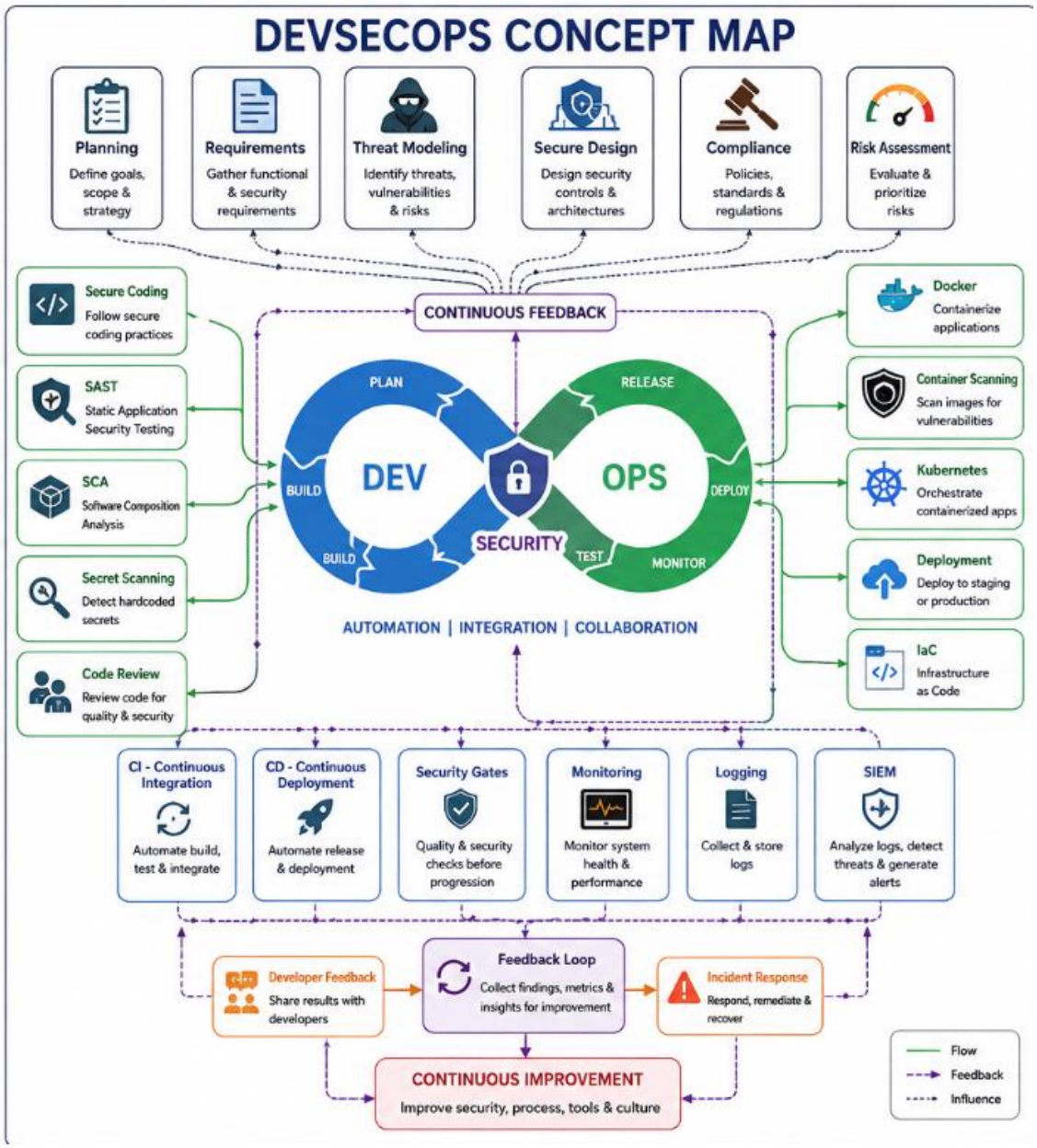
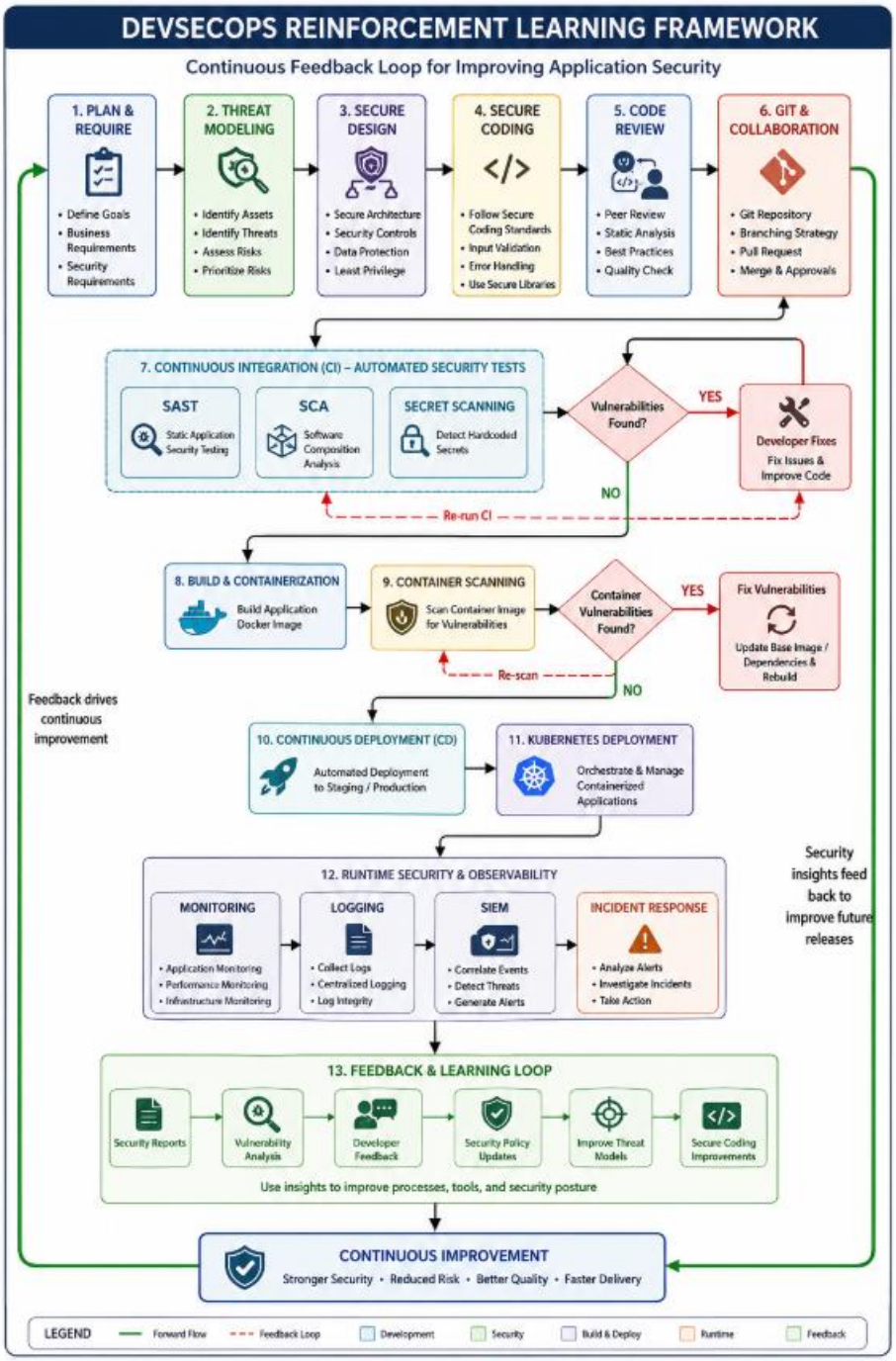
Assessment Task 1 (AT1) – Part 3

Concept Mapping, Reinforcement Learning Framework Visualization and GitHub Submission

CONCEPT MAP



Reinforcement Learning Framework Visualization



DevSecOps Concept Map and Reinforcement Learning Framework

1. Introduction

DevSecOps is a modern software development methodology that integrates security practices into every phase of the Software Development Life Cycle (SDLC). Unlike traditional development approaches where security is performed only at the final stage, DevSecOps ensures that security is continuously applied from planning to deployment and maintenance. It combines Development (Dev), Security (Sec), and Operations (Ops) into a single automated workflow that improves software quality, minimizes vulnerabilities, and accelerates secure software delivery.

This project presents a DevSecOps Concept Map illustrating the complete software delivery pipeline along with a Reinforcement Learning-inspired Framework Visualization that demonstrates how continuous feedback improves application security throughout the development lifecycle.

2. DevSecOps Concept Map Overview

The concept map represents the complete DevSecOps lifecycle by connecting all major phases involved in secure software development. The workflow begins with Planning and Requirements Analysis, where project objectives, business requirements, and security requirements are defined.

The next stage is Threat Modeling, where potential threats, vulnerabilities, and attack vectors are identified before software development begins. Based on these findings, Secure Design is performed to establish a secure architecture, apply security principles, and minimize potential risks.

Developers then implement the application using Secure Coding practices that follow secure coding standards, input validation, proper authentication, and secure error handling. During development, source code is managed using Git, while GitHub, Branches, and Pull Requests enable collaborative development and secure code reviews.

The code then enters the Continuous Integration (CI) pipeline where automated security tools perform Static Application Security Testing (SAST), Software Composition Analysis (SCA), and Secret Scanning. These automated checks identify coding vulnerabilities, insecure third-party libraries, and exposed credentials before deployment.

After successful validation, the application is packaged into Docker containers and subjected to Container Scanning to identify vulnerabilities within container images and operating system packages.

The validated application proceeds through the Continuous Deployment (CD) pipeline and is deployed into a Kubernetes environment for orchestration and management of containerized applications. Following deployment, Monitoring, Logging, and Security Information and

Event Management (SIEM) continuously observe application behavior, detect anomalies, and generate security alerts.

3. Security Integration Across the DevSecOps Lifecycle

Security is integrated into every stage of the development lifecycle rather than being treated as a separate process. Threat modeling reduces design risks before implementation begins. Secure coding practices minimize programming vulnerabilities during development. Automated security testing within the CI pipeline identifies issues immediately after code changes are committed.

Software Composition Analysis ensures that third-party libraries are free from known vulnerabilities, while Secret Scanning prevents accidental exposure of sensitive credentials such as API keys and passwords. Container Scanning validates Docker images before deployment to production.

Runtime security is maintained through continuous monitoring, centralized logging, and SIEM solutions that collect security events, correlate logs, detect suspicious activities, and generate alerts for incident response teams. This layered security approach enables early vulnerability detection and reduces the overall attack surface of the application.

4. Reinforcement Learning-Inspired Feedback Mechanism

The Reinforcement Learning Framework Visualization demonstrates a continuous feedback and improvement cycle within the DevSecOps pipeline. It does not implement a machine learning algorithm but adopts the concept of iterative learning through continuous security feedback.

During the Continuous Integration stage, automated security tools such as SAST, Software Composition Analysis (SCA), and Secret Scanning identify vulnerabilities in source code and project dependencies. If vulnerabilities are detected, developers immediately fix the issues, and the security tests are executed again until all checks are successfully completed.

Following successful builds, Docker container images undergo Container Scanning to detect operating system vulnerabilities and insecure packages. Any identified issues are resolved before deployment continues.

After deployment into Kubernetes, runtime monitoring tools continuously observe application performance, resource utilization, and security events. Centralized logging collects application and infrastructure logs, while SIEM platforms analyze these logs to detect threats, correlate security incidents, and generate alerts.

The security findings obtained from monitoring, logging, vulnerability analysis, and incident response are continuously shared with developers. These insights help improve secure coding

practices, update threat models, strengthen security policies, and enhance future software releases. This continuous feedback mechanism creates an iterative improvement loop that strengthens the organization's overall security posture over time.

5. Conclusion

The DevSecOps Concept Map and Reinforcement Learning Framework together demonstrate how security can be integrated into every phase of the software development lifecycle. Automated security testing, container security, continuous monitoring, and centralized logging help organizations identify and resolve vulnerabilities at the earliest possible stage.

The continuous feedback mechanism ensures that security findings are consistently used to improve coding practices, update security controls, and strengthen future software releases. By integrating automation, collaboration, and continuous security validation, DevSecOps enables organizations to deliver secure, reliable, and high-quality software while maintaining rapid development and deployment cycles.